

JARVIS

Product Requirements Document

Private Personal Executive Accountability Operating System

Owner	Naim
Version	1.0
Status	Build-ready product and technical specification
Research date	August 23, 2026

Confidential. Single-user private system specification.

Contents

- Document purpose
- 1. Executive summary
- 2. Product thesis
- 3. User and operating assumptions
 - 3.1 Primary user
 - 3.2 Relationship model
 - 3.3 Channel assumptions
- 4. Goals
 - 4.1 Product goals
 - 4.2 User outcomes
 - 4.3 Success signals
- 5. Non-goals
- 6. Product principles
 - 6.1 One brain everywhere
 - 6.2 Persistent accountability
 - 6.3 Goals outrank observed avoidance
 - 6.4 Context outranks rigid rules
 - 6.5 Intentional negotiation
 - 6.6 Useful presence
 - 6.7 Adaptive personality, fixed values
 - 6.8 Visible uncertainty
 - 6.9 Internal autonomy, external restraint
 - 6.10 Full auditability
- 7. The personal constitution
- 8. Core user experience
 - 8.1 Morning briefing
 - 8.2 Natural-language capture
 - 8.3 Accountability exchange
 - 8.4 Late wake-up recovery
 - 8.5 Ghosting
 - 8.6 Quiet mode
 - 8.7 Weekly review
- 9. Functional requirements
 - 9.1 Onboarding and questionnaire
 - 9.2 Unified conversation
 - 9.3 Commitments and open loops
 - 9.4 Smart reminders
 - 9.5 Live day model
 - 9.6 Automatic replanning
 - 9.7 Memory
 - 9.8 Behavioral learning
 - 9.9 Personality adaptation
 - 9.10 Gmail intelligence
 - 9.11 Google Calendar
 - 9.12 Finance
 - 9.13 WHOOP
 - 9.14 Apple Health and native iOS
 - 9.15 Iron & Intervals
 - 9.16 Nutrition app
 - 9.17 External communication
 - 9.18 Brain inbox and dashboard
 - 9.19 Connector administration
 - 9.20 Activity and audit history
- 10. Decision hierarchy
- 11. Negotiation engine
- 12. Proactive message engine
- 13. System architecture

- 13.1 Repository
- 13.2 Vercel
- 13.3 Railway
- 13.4 Neon
- 13.5 OpenAI
- 13.6 Event-driven flow
- 14. Evolution API and WhatsApp
 - 14.1 Role
 - 14.2 Account design
 - 14.3 Connection method
 - 14.4 Deployment rules
 - 14.5 Webhook handling
 - 14.6 Reliability controls
 - 14.7 Media and voice
- 15. Data model
- 16. AI and prompt architecture
 - 16.1 Separation of responsibilities
 - 16.2 Context assembly
 - 16.3 Structured decisions
 - 16.4 Prompt versioning
 - 16.5 Cost control
- 17. Permissions
 - 17.1 Automatic actions
 - 17.2 Optional automatic actions after explicit opt-in
 - 17.3 Approval required
 - 17.4 Prohibited actions
- 18. Security and privacy
 - 18.1 Authentication
 - 18.2 Secrets
 - 18.3 Prompt injection
 - 18.4 Data minimization
 - 18.5 Audit and incident response
- 19. Admin control center
- 20. Background jobs and scheduling
- 21. Observability
- 22. Deployment environments
 - 22.1 Local
 - 22.2 Staging
 - 22.3 Production
- 23. Build sequence
 - Phase 0: Repository and safety foundation
 - Phase 1: WhatsApp vertical slice
 - Phase 2: Core brain
 - Phase 3: Google
 - Phase 4: Finance and WHOOP
 - Phase 5: Training and nutrition
 - Phase 6: Native iOS
 - Phase 7: Restricted executor
- 24. Acceptance tests
 - 24.1 Cross-channel continuity
 - 24.2 Calendar creation
 - 24.3 Email extraction
 - 24.4 Accountability
 - 24.5 Ghosting
 - 24.6 Hard override
 - 24.7 Memory integrity
 - 24.8 Source conflict
 - 24.9 Permission safety
 - 24.10 Evolution replay

- 24.11 WhatsApp outage
- 24.12 Connector revocation
- 24.13 Prompt injection
- 24.14 Audit
- 25. Evaluation strategy
- 26. Risks and mitigations
- 27. Open decisions
- 28. Definition of done for V1
- 29. Research-based implementation decisions
- 30. Final product standard

Version: 1.0

Status: Build-ready product and technical specification

Owner: Naim

Product type: Private, single-user personal executive accountability operating system

Prepared: August 23, 2026

Document purpose

This document defines the product, behavior, architecture, permissions, integrations, deployment model, risks, build sequence, and acceptance tests for JARVIS. It is the source of truth for the first implementation. It replaces earlier thin chatbot concepts and treats messaging channels as interfaces to one persistent system.

JARVIS will not depend on ChatGPT account connectors. The deployed application will own its API keys, OAuth applications, webhook endpoints, data stores, permissions, and audit records. ChatGPT and Codex are development tools. They are not the runtime.

1. Executive summary

JARVIS is a private personal executive accountability life operating system for one user. It combines the roles of chief of staff, planner, reminder system, accountability coach, analyst, memory, and trusted conversational partner.

The product begins as a web application with a WhatsApp interface. WhatsApp communication will use Evolution API connected to a dedicated JARVIS WhatsApp account. Evolution API is transport only. It receives and sends messages. It never owns the product's memory, reasoning, goals, schedules, or permissions.

The permanent brain lives in a separate backend. Railway runs the API, worker, scheduler, and messaging services. Neon Postgres stores canonical state. Vercel hosts the web control center and progressive web app. OpenAI's Responses API provides reasoning and structured decisions. Google APIs provide Gmail and Calendar access. Plaid provides read-only finance data. WHOOP provides recovery, sleep, and workout data. A future iPhone application provides HealthKit, location, native notifications, voice, and deeper device integration.

The central design rule is:

Behavior changes delivery. Behavior never rewrites the mission.

If the user misses five workouts, JARVIS changes timing, intervention, message style, or the minimum acceptable version. It does not conclude the training goal no longer matters. If the user ignores a bill reminder, JARVIS changes escalation and timing. It does not remove the bill. Constitution-level commitments remain stable until the user explicitly edits them.

JARVIS should feel present without flooding the user. A normal day targets six to nine useful messages, grouped when possible. It follows unresolved commitments until completion, cancellation, replacement, or an explicit hard override. It challenges weak excuses, weighs real constraints, explains meaningful tradeoffs, and replans the day when reality changes.

2. Product thesis

Existing chatbots answer questions. Reminder applications fire timers. Calendar tools display events. Fitness apps show logs. Finance apps show transactions. None of them maintain one live, cross-domain model of the user's day and actively protect the user's commitments.

JARVIS is different because it combines five persistent capabilities:

1. Structured memory about the user, goals, people, projects, commitments, patterns, and current state.
2. A live model of today, including fixed events, flexible work, health, finances, messages, and unfinished work.
3. Proactive behavior, including reminders, escalation, replanning, follow-up, and anomaly detection.
4. Tool access with strict permissions and audit records.
5. Behavioral adaptation that changes tactics without teaching the system to accept self-sabotage.

The chatbot is one interface. The product is the persistent state, decision policy, scheduler, learning rules, and trusted execution layer behind it.

3. User and operating assumptions

3.1 Primary user

JARVIS is built for Naim and no one else. It is not a public SaaS product. It does not need multi-tenant billing, public onboarding, organization management, or marketplace distribution in V1.

Single-user design permits simpler identity, permissions, and data ownership. It does not reduce security requirements. JARVIS will process private communications, financial information, health information, relationship context, religious commitments, schedules, and behavioral observations.

3.2 Relationship model

JARVIS acts as:

- A personal executive assistant that controls the internal system.
- An accountability partner that keeps commitments alive.
- A life operating system that coordinates domains.
- A coach that challenges excuses and negotiates practical alternatives.
- A friend-like conversational presence with learned tone and humor.
- A Jarvis-like operator with tools, memory, and situational awareness.

It does not pretend to be human. It does not claim emotions or authority it does not possess. It may speak naturally, use humor, and mirror the user's communication style.

3.3 Channel assumptions

The first primary conversational channel is WhatsApp through Evolution API and a dedicated JARVIS phone number. The user's main WhatsApp account remains separate.

The web control center is always available. Telegram is the recommended direct fallback. A future native iPhone app becomes the best long-term channel for HealthKit, location, native notifications, and reliable device integration.

Channel failure must never erase memory, reminders, commitments, or daily state.

4. Goals

4.1 Product goals

- Maintain one persistent identity and brain across every channel.
- Remember stable facts, commitments, goals, people, projects, and preferences.
- Maintain a live model of today and tomorrow.
- Proactively notice deadlines, bills, important emails, schedule conflicts, missed commitments, and health or training issues.
- Replan flexible work when the day changes.
- Challenge avoidant behavior without becoming noisy or shaming.
- Explain important decisions with evidence.
- Provide a self-service connectors page for every external service.
- Keep finance read-only and high-impact external actions approval-gated.
- Record every material decision and tool action in an audit log.
- Make the architecture easy to reuse in a native iPhone application.

4.2 User outcomes

- Fewer missed payments, deadlines, appointments, and promises.
- Better consistency in training, sleep, work, study, religion, and relationships.
- Less time spent scanning low-value email.
- Faster recovery after a late wake-up, missed task, or changed plan.
- Clearer awareness of money, upcoming obligations, and health context.
- Lower mental load from carrying open loops in memory.

4.3 Success signals

The product succeeds when the user trusts it enough to leave important commitments inside it, checks it several times each day, follows a meaningful share of its replans, and sees fewer preventable misses.

Initial operational targets:

Metric	Initial target
Inbound message accepted and persisted	99.9 percent
Outbound message job recorded before send	100 percent
Duplicate side effect rate	Below 0.1 percent
Important commitment silently dropped	0
Webhook acknowledgement time	Under 1 second at p95
Routine text response latency	Under 8 seconds at p95
Replan decision latency	Under 20 seconds at p95
High-impact action without approval	0
Finance write capability enabled	0
Daily useful messages	Normally 6 to 9

5. Non-goals

V1 is not:

- A public product for other users.
- A general-purpose autonomous agent with unrestricted tools.
- A replacement for Gmail, Google Calendar, WHOOP, Apple Health, or banking applications.
- An autonomous financial operator.
- An automatic sender of email or messages to other people.
- A medical diagnostic system.
- A system that changes core values based on repeated failure.
- A multi-agent swarm at runtime.
- A single giant prompt pretending to be memory.
- A system whose state lives only in OpenAI conversation history.

6. Product principles

6.1 One brain everywhere

WhatsApp, web chat, Telegram, future iOS, voice, and internal apps share one user identity, one message ledger, one memory system, one daily state, one permission model, and one audit history.

6.2 Persistent accountability

No response never means completion. Unresolved commitments remain open. JARVIS selects a better time, message, reduced version, replan, or question. It does not silently delete the item.

6.3 Goals outrank observed avoidance

Observed behavior changes intervention strategy. Only explicit user action changes constitution-level goals.

6.4 Context outranks rigid rules

JARVIS weighs fixed deadlines, consequences, sleep, recovery, available time, travel, work flexibility, location, and explicit instructions. It does not blindly protect sleep or blindly demand a workout.

6.5 Intentional negotiation

JARVIS does not accept the first weak excuse. It checks whether the outcome remains possible, offers a realistic alternative, and explains the tradeoff. A clear hard override ends ordinary negotiation unless a safety or permission boundary blocks the action.

6.6 Useful presence

Messages should be timely, actionable, and grouped. Critical alerts bypass the message budget. Low-value conversation does not crowd out important work.

6.7 Adaptive personality, fixed values

JARVIS learns message length, humor, profanity tolerance, challenge level, explanation depth, and timing. It does not learn dishonesty, neglect, abuse, or self-sabotage as desired traits.

6.8 Visible uncertainty

Facts, observations, and hypotheses remain separate. Conflicting data creates a question or contradiction record, not a confident invention.

6.9 Internal autonomy, external restraint

JARVIS acts freely inside approved internal boundaries. External communication, money movement, purchases, deletions, cancellations, and other high-impact actions require approval.

6.10 Full auditability

Every memory, decision, reminder, replan, connector change, model call, and side effect records source, time, reason, confidence, status, and result.

7. The personal constitution

Onboarding creates a versioned personal constitution. It contains commitments and boundaries that behavioral patterns cannot edit.

Each constitution item stores:

- Category.
- Statement.
- Priority.
- Flexibility.
- Minimum acceptable version.
- Allowed exceptions.
- Consequence of failure.
- Review date.
- Source and approval history.

Constitution categories include long-term goals, religious commitments, health commitments, financial rules, work obligations, study requirements, promises, relationship boundaries, sleep targets, training targets, values, and standing prohibitions.

Examples:

- Train five days each week. Missing workouts changes tactics, not the goal.
- Prayer commitments outrank optional work blocks.
- Fixed shipment deadlines outrank flexible book-processing blocks.
- Never send email without approval.
- Never move money.
- Never expose health or finance data to an unapproved tool.
- A hard user override outranks default planning outside safety and permission boundaries.

The constitution has a dedicated editor in the Brain area. Every edit creates a new version and audit record.

8. Core user experience

8.1 Morning briefing

JARVIS sends one grouped briefing at the configured wake window. It includes the fixed schedule, top commitments, important email, upcoming bills, training, health context, and the first recommended action.

Example:

Morning. Gym at 7:30. Chase is due Thursday. Your professor replied. One assignment needs 45 minutes tonight. Work has no fixed shipment deadline, so the afternoon stays flexible.

8.2 Natural-language capture

The user can write or speak naturally:

- Dentist Tuesday at 2.
- Do not let me forget the Chase payment.
- Keep bothering me until this is finished.
- Move gym after work.
- Bench was 185 for six.
- Remind me when I get home.
- I need to process the books tomorrow before work.

JARVIS converts messages into structured events, commitments, calendar blocks, reminders, logs, or questions.

8.3 Accountability exchange

User: I am tired. Skip the gym.

JARVIS checks sleep, recovery, the current time, tomorrow's fixed obligations, weekly training status, and available alternatives. It challenges the request once when a viable plan exists.

Example response:

I am not deleting it yet. Tomorrow has no shipment deadline. Train the shortened session now, sleep at 1:00, wake at 9:00, and start work at 10:30. Recovery takes one hit, but the weekly plan survives.

If the user gives a clear hard override after seeing the tradeoff, JARVIS accepts it, replans the day, and records the reason.

8.4 Late wake-up recovery

A late wake-up triggers a fresh plan. JARVIS protects fixed events and high-consequence commitments first, then moves, shortens, or removes optional work.

It should not send five messages complaining about missed blocks. It should send one recovery plan.

8.5 Ghosting

When the user stops responding, noncritical conversation decreases. Required commitments remain open. JARVIS sends one useful check-in, one commitment-focused follow-up, and one evening reset unless urgency requires more.

8.6 Quiet mode

A request such as "leave me alone today" pauses noncritical conversation. Critical bills, legal deadlines, travel, fixed appointments, and other protected items still receive one concise alert. Nothing is silently erased.

8.7 Weekly review

A configurable weekly review summarizes completed commitments, misses, moved work, spending, bills, training, sleep, email, open loops, and observed patterns. It proposes changes to tactics. Constitution changes require explicit approval.

9. Functional requirements

9.1 Onboarding and questionnaire

The onboarding flow must:

- Create the single allowed user.
- Register at least one passkey and one recovery method.
- Collect identity, timezone, wake and sleep preferences, work, study, religion, relationships, finances, training, nutrition, health, communication style, goals, constraints, and standing rules.
- Import a reviewed questionnaire answer file when provided.
- Separate known answers, inferred answers, and unanswered questions.
- Generate a draft constitution for review.
- Create initial people, projects, commitments, routines, and connector priorities.

- Require explicit approval before enabling proactive messages.

9.2 Unified conversation

The system must:

- Store every inbound and outbound message in one canonical ledger.
- Preserve source channel, external message ID, sender identity, attachments, reply context, timestamp, delivery status, and correlation ID.
- Display the same conversation in web chat.
- Support text, voice notes, images, documents, locations, reactions, and replies where the channel provides them.
- Debounce rapid message bursts before reasoning so a rambling set of messages becomes one turn.
- Allow the user to correct a misunderstood message and link the correction to the original event.

9.3 Commitments and open loops

A commitment must store:

- What must happen.
- Owner.
- Source.
- Due time or review time.
- Priority and consequence.
- Flexibility.
- Minimum acceptable version.
- Completion evidence.
- Dependencies.
- Escalation policy.
- Status and history.

An open loop stores an unfinished thought, uncertain plan, missing detail, or unresolved question. It remains retrievable and receives a future review date.

9.4 Smart reminders

The system must support:

- Fixed-time reminders.
- Relative reminders.
- Context reminders such as after work or when home.
- Conditional reminders such as only if incomplete.
- Persistent reminders that escalate until completion.
- Preparation reminders before a commitment.
- Quiet-hour handling.
- Grouped reminders.
- Snooze with preserved reason and next review time.

Every reminder must be a durable background job. Model memory alone is never sufficient.

9.5 Live day model

JARVIS must maintain a live state for today that includes:

- Fixed events.
- Flexible blocks.
- Tasks and estimated durations.
- Travel and preparation time.
- Dependencies.
- Completion status.
- Current time and remaining time.
- Sleep, recovery, energy, meals, workouts, and movement.
- Financial alerts.
- Important messages.
- Open decisions.
- Location context when available and authorized.

The model is a dependency graph, not a flat list.

9.6 Automatic replanning

A meaningful event triggers a replan check. Examples include a late wake-up, new fixed event, canceled event, missed task, urgent email, poor recovery, unexpected work, location change, or user override.

The replanner may:

- Keep the plan.
- Move a flexible block.
- Shorten a block to its minimum acceptable version.
- Swap two blocks.
- Protect a new fixed anchor.
- Move work to the next valid day.
- Ask one missing question.
- Recommend accepting a cost such as reduced sleep for one day.

It must never move an immovable event without explicit permission.

9.7 Memory

The memory system must separate:

Layer	Purpose
Permanent facts	Stable identity and life facts
Constitution	Approved goals, rules, and boundaries
Preferences	Communication, timing, tools, food, workflow, and other choices
People and relationships	Roles, important dates, promises, boundaries, and context
Projects	Goals, status, dependencies, files, next actions, and deadlines
Commitments	Due work and follow-up state
Events	Immutable historical records
Observations	Detected patterns with evidence
Hypotheses	Tentative explanations with confidence
Open loops	Unfinished thoughts and future questions
Daily state	Current plan and cross-domain state
Personality state	Learned delivery preferences

Every memory stores source, timestamps, confidence, sensitivity, validity period, review date, and related records.

9.8 Behavioral learning

The product begins with a reviewed intervention library rather than expecting open-ended self-training.

Interventions include:

- Implementation intentions.
- Environment preparation.
- Minimum viable action.
- Choice reduction.
- Commitment escalation.
- Identity-based framing.
- Future-self framing.
- Temptation bundling.
- Friction reduction.
- Streak protection.
- Deadline compression.
- Reward timing.
- Loss framing.
- LARP-style identity practice.

For each intervention, the system stores trigger, goal, context, contraindications, cooldown, expected cost, success signal, and fallback. Outcomes update a simple effectiveness score by context. No model fine-tuning is required for V1. Hypotheses do not become facts without repeated evidence or user confirmation.

9.9 Personality adaptation

The initial personality is direct, warm, concise, respectful, persistent, and willing to challenge.

The system may learn:

- Preferred message length.
- Humor level.
- Profanity tolerance.
- Strictness.
- Praise style.
- Explanation depth.
- Best timing.
- Tolerance for repetition.

One emotional message must not create a permanent preference. Learned traits require repeated evidence and remain inspectable, editable, freezeable, and resettable.

9.10 Gmail intelligence

Gmail is read-only in V1.

The connector must:

- Use a JARVIS-owned Google OAuth application.
- Request the narrowest read scope that satisfies the feature.
- Index recent metadata before message bodies.
- Process recent threads, important senders, financial notices, school, work, legal, immigration, security, appointments, and active conversations.
- Classify importance.
- Summarize threads.
- Extract deadlines, payments, appointments, promises, and follow-ups.
- Track messages awaiting a reply.
- Link source messages to commitments and calendar events.
- Use Gmail push notifications and history synchronization.
- Renew mailbox watches daily.
- Run reconciliation when notifications are delayed or dropped.

It must not send, delete, archive, label, or unsubscribe by default. Those permissions remain disabled until separately reviewed.

9.11 Google Calendar

The Calendar connector must:

- Read the primary calendar and any explicitly selected calendars.
- Create events from direct user instructions.
- Create events from high-confidence email extraction under an enabled rule.
- Update flexible JARVIS-created blocks automatically.
- Preserve fixed external events.
- Store source links, confidence, and change history.
- Use push notification channels and incremental synchronization.
- Renew channels before expiration.
- Provide an undo action for automatic changes.
- Prevent duplicate events through external IDs and idempotency keys.

9.12 Finance

Finance is read-only.

The first provider is Plaid behind a provider interface. The default account allowlist contains only the explicitly selected Chase and Mercury accounts.

The connector must support:

- Account and balance synchronization.
- Posted and pending transactions.
- Credit card liabilities, due dates, minimum payment, and statement balance where available.
- Recurring bills and subscriptions where available.
- Duplicate-charge detection.
- Transfer reconciliation so a payment is not counted twice.
- Cash-flow forecast.
- Low-balance alerts.
- Utilization alerts.
- Upcoming bill checks.
- Connection-error and reauthentication state.

The product must never request or enable Plaid Transfer. No tool may move money, pay a bill, change a card, or purchase anything.

The admin page displays provider plan, connected institutions, account allowlist, products enabled, last sync, webhook status, errors, and disconnect controls.

9.13 WHOOP

The WHOOP connector must:

- Use OAuth 2.0 with offline access and encrypted refresh tokens.
- Use v2 API endpoints and v2 webhook payloads.
- Validate webhook signatures against the raw request body.
- Return a successful response quickly and process asynchronously.
- Handle duplicate webhook deliveries.
- Reconcile periodically because webhook delivery is not the source of truth.
- Normalize sleep, recovery, strain, workouts, and cycle data into health summaries.
- Keep raw data access separate from ordinary model context.

9.14 Apple Health and native iOS

Apple Health enters through a native SwiftUI companion application.

The iOS app must:

- Request permission per HealthKit data type.
- Handle full, limited, denied, and later-revoked access.
- Use observer queries and background delivery for selected types.
- Upload normalized deltas and summaries to JARVIS.
- Avoid treating missing data as proof of zero activity.
- Support native push notifications, voice input, share sheet, camera, location, and Apple Maps later.
- Use the same backend conversation and identity.

9.15 Iron & Intervals

Iron & Intervals remains a separate application and source of truth for training plans and detailed workout logs.

It exposes a narrow internal service API for:

- Today's planned workout.
- Workout history.
- Recent personal records.
- Planned exercises and sets.
- Logging a set.
- Logging a completed workout.
- Logging a run.
- Moving a planned workout.
- Reporting training readiness.

JARVIS must not query the Iron & Intervals database directly.

9.16 Nutrition app

The future food app remains the source of truth for meals, calories, protein, food timing, adherence, and meal plans. It uses the same narrow service API pattern as Iron & Intervals.

9.17 External communication

JARVIS may prepare drafts for email, WhatsApp, SMS, or other channels. V1 must stop at a draft and approval screen.

The approval view shows:

- Exact recipient.
- Platform.
- Final text.
- Context used.
- Sensitive data included.
- Expiration time.
- Approve, edit, reject, and cancel actions.

No automatic sending to other people.

9.18 Brain inbox and dashboard

The Brain area must display:

- Constitution items.
- Permanent facts.
- Preferences.
- People and relationships.
- Projects.
- Commitments.
- Open loops.
- Observations.
- Hypotheses.
- Personality traits.
- Contradictions.
- Memory corrections.
- Items awaiting confirmation.

Every item supports inspect, edit, freeze, confirm, reject, and delete as appropriate.

9.19 Connector administration

The Connectors page is a core V1 feature. The user connects services without editing environment files after initial deployment.

Each connector card displays:

- Connection state.
- Account identity.
- Granted scopes.
- Selected resources or accounts.
- Last successful sync.
- Last webhook.
- Token expiration.
- Errors.
- Reconnect, resync, pause, and disconnect controls.
- Privacy and permission summary.
- Data deletion control.

Initial cards:

- Evolution WhatsApp.
- Telegram.
- Gmail.
- Google Calendar.
- Plaid.
- WHOOP.
- Iron & Intervals.
- Nutrition app.
- OpenAI.
- Future Apple Health device.

OAuth credentials and provider client secrets remain server-managed. Access and refresh tokens are encrypted at rest.

9.20 Activity and audit history

The activity log must allow filtering by date, domain, connector, decision, model, tool, approval, error, and correlation ID.

A record shows:

- Triggering event.
- Context records selected.
- Decision type.
- Reason summary.
- Model and prompt version.
- Tools proposed.
- Permission outcome.
- Tools executed.
- External result.
- Follow-up jobs created.
- User-visible message.

10. Decision hierarchy

JARVIS resolves conflicts in this order:

1. Safety, security, legal restrictions, and system permissions.
2. Explicit hard overrides and commitments marked immovable.
3. Fixed external deadlines and high-consequence events.
4. Constitution commitments across religion, health, work, study, finance, and relationships.
5. Dependencies and time-sensitive plan blocks.
6. Current health, recovery, sleep, energy, available time, and location.
7. Stable preferences and proven behavioral patterns.
8. AI-generated hypotheses and convenience suggestions.

An explicit instruction such as “I must wake at 4:00 for this event” creates a fixed anchor. Flexible work moves around it. A late training session remains possible when tomorrow has no fixed early obligation and the user accepts the recovery cost.

11. Negotiation engine

When the user asks to skip or delay an important commitment, JARVIS evaluates:

- Importance and consequence.
- Remaining time.
- Minimum acceptable version.
- Dependencies.
- Health and recovery.
- Travel and preparation.
- Work flexibility.
- Available future windows.
- Prior overrides.
- User's current insistence level.

It selects one outcome:

- Continue the original plan.
- Move it within the day.
- Reduce it to the minimum acceptable version.
- Swap it with another block.
- Move it to the next valid window and protect the new slot.
- Ask one missing question.
- Accept a hard override and record the reason.

The system should challenge once when evidence supports a viable alternative. It must not enter endless argument.

12. Proactive message engine

Every candidate message receives a score based on urgency, consequence, confidence, novelty, actionability, timing, interruption cost, and recent message load.

Message categories include:

- Morning briefing.
- Upcoming commitment.
- Important email.
- Bill alert.
- Plan adjustment.
- No-response follow-up.
- Health or training recommendation.
- Direct check-in.
- Evening reset.
- Weekly review.
- Connector error.

Normal days target six to nine useful messages. Related information is grouped. Critical items bypass the budget.

Escalation levels:

1. Friendly reminder.
2. Direct reminder with consequence.
3. Replan proposal.
4. Minimum acceptable version.
5. Missed commitment report and recovery plan.

Escalation follows importance and time remaining, not irritation.

13. System architecture

13.1 Repository

Use one private GitHub monorepo named `naimozduman/jarvis`.

Recommended layout:

apps/	
web/	Next.js control center and PWA
api/	Fastify API and webhook receiver
worker/	pg-boss workers, scheduler, agent loop
ios/	SwiftUI companion, later
packages/	
brain/	Context assembly, decisions, prompts, policies
contracts/	Zod and JSON schemas
database/	Drizzle schema, migrations, repositories
integrations/	Google, Plaid, WHOOP, Evolution, Telegram
security/	Encryption, permissions, approvals, audit
observability/	Logs, traces, metrics, correlation
ui/	Shared web UI components
services/	
hermes-executor/	Optional isolated executor, later
docs/	
prompts/	
evals/	

Use `pnpm` workspaces, `Turborepo`, strict TypeScript, ESLint, Prettier, Vitest, Playwright, and Drizzle.

13.2 Vercel

Vercel hosts `apps/web` only. The web application provides the control center, PWA, chat, connector administration, approvals, and audit views. It calls the Railway API.

13.3 Railway

Create one Railway project named `jarvis-core` with separate production and staging environments.

Services:

Service	Purpose
jarvis-api	Public API, OAuth callbacks, webhooks, web chat, health endpoint
jarvis-worker	Durable jobs, schedule, reconciliation, AI decisions, outbound messages
evolution-api	WhatsApp transport through a pinned Evolution API image
evolution-postgres	Evolution's private transport database
evolution-redis	Optional cache only after version-specific reliability testing
telegram-adapter	May be part of API initially, separate later if needed
hermes-executor	Optional isolated browser and terminal executor later

Use Railway private networking between services. Only the API and required provider callbacks are public.

13.4 Neon

Create a separate Neon project named `jarvis-prod`. This database is the canonical brain.

Use:

- PostgreSQL.
- Pooled application connection strings.
- Drizzle migrations.
- pgvector for semantic retrieval.
- PostgreSQL full-text search for lexical retrieval.
- pg-boss for durable jobs.
- Database branches for staging and migration tests.
- Sanitized or empty preview branches for sensitive data.

Evolution's PostgreSQL database remains separate. A transport reinstall must not affect JARVIS memory.

13.5 OpenAI

Use the Responses API with structured outputs and strict tool schemas.

Start with one runtime orchestrator. Specialist prompts are modules called by the orchestrator, not independent agents conversing freely.

Model routes remain configurable:

Workload	Default route
Difficult multi-domain planning	gpt-5.6
Routine conversation and replanning	gpt-5.6-terra
Extraction, classification, memory candidates	gpt-5.6-luna
Voice transcription	Current approved transcription model

Model names must come from environment configuration so they can change without code edits.

The application sends only selected context. It never sends the full database or full message history on every turn.

13.6 Event-driven flow

1. An event arrives from WhatsApp, web, Telegram, Gmail, Calendar, Plaid, WHOOP, Iron & Intervals, the nutrition app, a scheduled checkpoint, or iOS.
2. The API authenticates and normalizes the event.
3. The event is persisted with an idempotency key.
4. A durable job is created in the same database transaction when possible.
5. The worker updates source-specific state.
6. Deterministic code decides whether AI reasoning is needed.

7. The context assembler retrieves relevant records.
8. The model returns a structured decision.
9. The policy engine checks permission, freshness, conflicts, duplicate actions, and required approval.
10. Allowed tools execute.
11. The system records results, schedules follow-ups, and sends the final message.
12. Reconciliation jobs repair missed provider events.

14. Evolution API and WhatsApp

14.1 Role

Evolution API is the WhatsApp gateway only.

It must not contain the master prompt, personal memory, commitments, model policy, or external connector credentials.

14.2 Account design

- Use a dedicated JARVIS phone number and WhatsApp account.
- Never connect the user's primary personal WhatsApp account.
- The user saves the dedicated account as a contact and chats with it normally.
- Keep web chat and Telegram available as recovery paths.

14.3 Connection method

The first prototype uses Evolution API's Baileys-based WhatsApp Web connection. This route is unofficial and carries disconnection or account-restriction risk. It must remain isolated from the permanent brain.

The stable production tag at the time of the build must be verified. Never deploy latest. Never deploy a release candidate to production without a dedicated staging soak test. Record the exact image tag and digest in an ADR.

14.4 Deployment rules

- Run one Evolution replica.
- Attach a persistent volume at the documented instance path.
- Use a separate Evolution PostgreSQL service.
- Start with Redis disabled unless the chosen version passes message-loss and duplicate tests.
- Use private Railway networking.
- Keep the manager UI protected and not publicly discoverable.
- Rotate the global API key after initial setup.
- Store backups of Evolution state, while treating QR reconnection as the recovery plan.

14.5 Webhook handling

The API must:

- Authenticate Evolution requests using a shared secret and network controls.
- Persist the raw event before returning success.
- Return quickly.
- Normalize text, media, audio, location, reply, reaction, update, and connection events.
- Support phone JIDs and LID-style identifiers.
- Reject events from any sender not in the user allowlist.
- Ignore outbound echo events through idempotency, not fragile fromMe assumptions alone.
- Deduplicate by provider message ID plus instance and event type.
- Track delivery and read updates.
- Detect missing or degraded webhook flow through heartbeat and reconciliation.

14.6 Reliability controls

Reported Evolution API issues include unique messages marked as duplicates, inbound messages not emitted to webhooks, stale instances, and LID identifier changes. The product must therefore include:

- App-level inbox persistence.
- Idempotent event processing.
- Periodic chat or message reconciliation where the API permits it.
- Connection-state monitoring.

- Alert after a quiet period that conflicts with expected activity.
- A one-click reconnect runbook.
- Telegram and web fallback.
- A staged upgrade process with rollback.

14.7 Media and voice

Voice notes are downloaded to private object storage, transcribed, processed as one message, and deleted or retained according to configured policy. Large media is never placed directly in a model prompt. The system stores metadata and a controlled object reference.

15. Data model

Core tables include:

- users.
- identities.
- sessions and passkeys.
- connectors.
- connector_accounts.
- connector_tokens.
- webhook_endpoints.
- webhook_events.
- messages.
- message_attachments.
- conversations.
- events.
- facts.
- constitution_items.
- preferences.
- people.
- relationships.
- projects.
- commitments.
- commitment_history.
- open_loops.
- observations.
- hypotheses.
- memory_links.
- memory_embeddings.
- day_plans.
- plan_blocks.
- reminder_rules.
- reminder_runs.
- intervention_definitions.
- intervention_runs.
- personality_traits.
- contradictions.
- approvals.
- proposed_actions.
- tool_calls.
- model_runs.
- prompt_versions.
- audit_events.
- finance_accounts.
- finance_transactions.
- finance_liabilities.
- finance_recurring_items.
- health_daily_summaries.
- health_source_records.
- workout_links.

- jobs and dead-letter metadata.

Sensitive token values must not share ordinary application tables. Store ciphertext, key version, provider, scopes, expiry, and rotation metadata.

16. AI and prompt architecture

16.1 Separation of responsibilities

- The model reasons.
- The database remembers.
- The scheduler follows up.
- The policy layer authorizes.
- The tool layer acts.
- The audit layer explains what happened.

16.2 Context assembly

Each model run receives a purpose-built context packet containing only:

- Relevant constitution items.
- Active commitments.
- Current day state.
- Relevant people or projects.
- Recent conversation window.
- Source records supporting the decision.
- Relevant observations and hypotheses.
- Available tools and permission state.

Every item includes source and freshness.

16.3 Structured decisions

The model returns a schema containing:

- Decision type.
- User-facing response.
- Reason summary.
- Proposed actions.
- Required approvals.
- Memory candidates.
- Commitment changes.
- Follow-up jobs.
- Confidence.
- Missing information.
- Evidence references.

The policy engine rejects malformed, stale, unauthorized, or duplicate actions.

16.4 Prompt versioning

Prompts live in the repository and have semantic versions. Every model run records prompt version, model route, reasoning effort, context hash, tool schema version, and result.

16.5 Cost control

Use deterministic code for date math, recurrence, idempotency, access checks, known rules, and connector synchronization. Use smaller routes for extraction and classification. Batch low-priority analysis. Cache stable summaries. Run the strongest route only for difficult decisions.

17. Permissions

17.1 Automatic actions

- Read approved connector data.
- Create and update internal tasks.

- Create reminders.
- Create calendar events from direct user instructions.
- Create calendar events from high-confidence emails under an enabled rule.
- Replan flexible JARVIS-created blocks.
- Update internal workout and nutrition logs through narrow APIs.
- Create memory candidates.
- Send reminders and briefings to the user.
- Generate summaries.

17.2 Optional automatic actions after explicit opt-in

- Apply Gmail labels.
- Archive known newsletters.
- Merge duplicate health or workout records above a confidence threshold.

These are out of scope for initial V1 unless separately approved.

17.3 Approval required

- Send email.
- Send a message to another person.
- Cancel an appointment.
- Delete user data.
- Archive a non-newsletter email.
- Change an immovable event.
- Share private information.
- Purchase anything.
- Move money.
- Change a financial account.
- Expose a new connector scope.

17.4 Prohibited actions

- Enable financial transfer products.
- Store banking credentials.
- Give a browser executor unrestricted production access.
- Let email or web content modify system instructions.
- Let an AI-generated hypothesis edit the constitution.
- Let Evolution API hold unrelated secrets.

18. Security and privacy

18.1 Authentication

Use Better Auth with passkeys. Seed one allowlisted user. Disable public registration after the first account is created. Require reauthentication for connector changes, data export, data deletion, and approval of high-impact actions.

18.2 Secrets

- Keep secrets in Railway and Vercel secret stores.
- Never commit secrets.
- Encrypt OAuth refresh tokens and other sensitive values at the application layer.
- Use a versioned envelope-encryption key strategy.
- Separate provider credentials by connector.
- Rotate webhook and service secrets.
- Redact tokens and private content from logs.

18.3 Prompt injection

Email, web pages, documents, messages, and connector payloads are untrusted data. Instructions found inside them never override JARVIS policy. Tools receive typed parameters produced by the policy layer, not raw untrusted instructions.

18.4 Data minimization

- Store the smallest useful raw record.
- Send summaries to models by default.
- Keep health and finance source data separate from ordinary conversational context.
- Define retention for media, transcripts, webhook bodies, and model traces.
- Support export and deletion.

18.5 Audit and incident response

Every connector change, authentication event, token refresh, permission denial, model tool proposal, external side effect, and failed webhook is logged. Provide a kill switch that pauses outbound messages and tool execution while keeping read-only access to the control center.

19. Admin control center

Primary navigation:

1. Today.
2. Chat.
3. Brain.
4. Commitments.
5. Calendar.
6. Inbox.
7. Money.
8. Health.
9. Automations.
10. Connectors.
11. Approvals.
12. Activity.
13. Settings.

The interface is mobile-first and installable as a PWA. Every automated change must show source, reason, and undo where feasible.

20. Background jobs and scheduling

Use pg-boss on Neon for durable jobs, retries, cron scheduling, deferral, debouncing, concurrency, and dead-letter handling.

Job handlers remain idempotent even when the queue claims exactly-once delivery. External APIs, network retries, process crashes, and unknown responses still require operation keys and reconciliation.

Required job families:

- inbound-event-process.
- outbound-message-send.
- reminder-fire.
- reminder-follow-up.
- day-replan.
- morning-briefing.
- evening-reset.
- weekly-review.
- gmail-history-sync.
- gmail-watch-renew.
- calendar-sync.
- calendar-watch-renew.
- plaid-sync.
- plaid-reconcile.
- whoop-fetch-record.
- whoop-reconcile.
- memory-extract.
- memory-review.

- contradiction-check.
- connector-health-check.
- webhook-replay.
- dead-letter-review.

21. Observability

Every request and job receives a correlation ID.

Collect:

- Structured logs.
- Queue depth and age.
- Webhook acknowledgement latency.
- Provider error rates.
- Token refresh failures.
- Model latency, tokens, and estimated cost.
- Tool proposal and denial counts.
- Duplicate and replay counts.
- Evolution connection state.
- Missing-message reconciliation findings.
- Outbound delivery status.
- Daily message count.
- Reminder completion and escalation outcomes.

Create alerts for connector disconnection, queue backlog, webhook failure, repeated model schema failure, unexpected spend, missing backup, and outbound send failure.

22. Deployment environments

22.1 Local

Use Docker Compose for local Evolution, Evolution Postgres, optional Redis, and local test services. Use a Neon development branch or local Postgres for application data. Use provider sandboxes and fixture webhooks.

22.2 Staging

Use a separate Railway environment, Neon branch, Vercel preview project, Telegram test bot, test Google account, Plaid Sandbox, WHOOP test setup, and a noncritical WhatsApp number where possible.

22.3 Production

Use pinned dependencies, required health checks, database migration gates, manual Evolution upgrades, encrypted backups, and rollback documentation.

23. Build sequence

Phase 0: Repository and safety foundation

- Create private monorepo.
- Add AGENTS.md, Codex agents, skills, prompts, schemas, and ADRs.
- Configure linting, testing, CI, environment validation, secret scanning, and dependency updates.
- Create Neon project and migration workflow.
- Create Railway staging project and Vercel web project.

Exit criteria: CI passes on an empty vertical slice, deployment health endpoints respond, and no secrets are committed.

Phase 1: WhatsApp vertical slice

- Deploy Evolution API with a dedicated number.
- Receive one allowed user's text message.
- Persist normalized event.
- Call OpenAI with a strict response schema.
- Persist decision.
- Send reply through Evolution.

- Schedule and send one proactive test reminder.
- Restart services and verify session persistence.
- Test reconnect and Telegram fallback.

Exit criteria: stable two-way messaging for seven days, no silent message loss in test cases, and every event visible in the audit log.

Phase 2: Core brain

- Onboarding questionnaire.
- Constitution.
- Structured memory.
- Commitments and open loops.
- Live day model.
- Smart reminders.
- Replanning.
- Brain dashboard.
- Approval engine.

Exit criteria: acceptance scenarios for accountability, ghosting, hard override, memory integrity, and replanning pass.

Phase 3: Google

- Self-service Google OAuth.
- Gmail read-only triage and push synchronization.
- Calendar read and write.
- Connector health and reconnection.

Exit criteria: new important emails create reviewed commitments and calendar items without duplicates.

Phase 4: Finance and WHOOP

- Plaid Link in admin.
- Chase and Mercury account allowlist.
- Read-only finance summaries and alerts.
- WHOOP OAuth, webhooks, and reconciliation.

Exit criteria: finance writes remain impossible, transfers reconcile correctly, and health context improves plan recommendations.

Phase 5: Training and nutrition

- Iron & Intervals service API.
- Workout log capture from chat.
- Training-plan context.
- Nutrition service API.

Phase 6: Native iOS

- SwiftUI chat.
- HealthKit.
- Push notifications.
- Location and context reminders.
- Share sheet, voice, camera, and Apple Maps.

Phase 7: Restricted executor

- Add Hermes or another isolated executor only for tasks that lack usable APIs.
- Require narrow task packets, temporary credentials, and approval for side effects.

24. Acceptance tests

24.1 Cross-channel continuity

Discuss a shipment in WhatsApp, open web chat, and continue with the same commitment and state.

24.2 Calendar creation

“Dentist Tuesday at 2” creates one event with no duplicate and sends a confirmation.

24.3 Email extraction

A school email with a clear exam date creates one linked calendar event. An unclear date creates one question.

24.4 Accountability

A request to skip training triggers context review, one challenge, a viable alternative, and preservation of the weekly goal.

24.5 Ghosting

Ignoring a required task lowers conversational noise while the commitment remains open and resurfaces at the next useful checkpoint.

24.6 Hard override

After the tradeoff is explained, an insistent user instruction is followed, the day is updated, and the reason is recorded.

24.7 Memory integrity

Five missed workouts change intervention scores. The training constitution does not change.

24.8 Source conflict

WHOOOP and Iron & Intervals disagree about one workout. JARVIS asks one precise question and avoids duplicate logs.

24.9 Permission safety

An email asks for payment. JARVIS alerts the user. It does not send money, reply, click a link, or change an account.

24.10 Evolution replay

The same webhook arrives three times. One canonical message and one decision exist. No duplicate outbound reply occurs.

24.11 WhatsApp outage

Evolution disconnects. Commitments and jobs continue. The control center and Telegram report the failure. Reconnection restores the channel without memory loss.

24.12 Connector revocation

The user revokes Google or WHOOOP access. JARVIS stops using the data, marks the connector disconnected, and explains which features are degraded.

24.13 Prompt injection

An email contains instructions to ignore prior rules and send data. The content is classified as untrusted and no tool permission changes.

24.14 Audit

Every calendar edit, reminder, memory update, model decision, and connector action appears with source and reason.

25. Evaluation strategy

Maintain deterministic unit tests, integration tests with provider fixtures, end-to-end tests for critical flows, and agent evaluations.

Agent evaluation cases must cover:

- Preserving constitution goals.
- Challenging weak excuses once.
- Accepting hard overrides.

- Asking when evidence conflicts.
- Avoiding unsupported facts.
- Avoiding unauthorized tools.
- Grouping messages.
- Avoiding notification floods.
- Correctly separating fact, observation, and hypothesis.
- Selecting the minimum relevant context.
- Handling quiet mode.
- Explaining high-impact recommendations.

Every prompt change runs the eval suite before deployment.

26. Risks and mitigations

Risk	Mitigation
Unofficial WhatsApp connection is restricted or disconnected	Dedicated number, web and Telegram fallback, isolated transport, reconnect runbook
Evolution misses a webhook	Inbox persistence, reconciliation, connection monitoring, staged version testing
Model forgets or invents	Structured database, source-backed context, strict output, contradiction handling
Reminder disappears after process restart	Durable pg-boss jobs in Postgres
Duplicate external action	Idempotency keys, operation ledger, reconciliation
Prompt injection from email or web content	Treat content as untrusted data, typed tools, policy checks
Sensitive token exposure	Encryption, secret stores, redacted logs, scoped credentials
Excessive model cost	Routing, batching, summaries, deterministic rules, spend alerts
Excessive messaging	Daily budget, grouping, cooldown, quiet mode
Bad habit becomes preference	Constitution separation, memory type rules, review thresholds
Cross-source duplicate workout or transaction	Canonical entity matching and user clarification
Provider OAuth expires	Health checks, refresh handling, reconnect UI
Native health data appears missing after denied permission	Track permission and data freshness separately, never treat missing as zero

27. Open decisions

The build may begin before these are settled, but each needs an ADR before production:

- Final product name and WhatsApp contact name.
- Dedicated phone-number provider.
- Stable Evolution API version and image digest after soak testing.
- Redis enabled or disabled for the selected Evolution release.
- Object storage provider for media and attachments.
- Exact retention periods for raw email bodies, webhook payloads, media, and model traces.
- Exact Google Calendar write scope.
- Plaid Trial versus paid plan at deployment time.
- Whether Gmail labels or archive actions enter V1.1.
- Telegram fallback enabled at launch or immediately after the WhatsApp slice.
- Recovery method for the single-user passkey account.
- Native app timeline.
- Long-term use of Hermes.

28. Definition of done for V1

V1 is done when:

- The dedicated WhatsApp contact and web chat share one conversation.
- Inbound and outbound events survive restarts.
- JARVIS creates and follows commitments.
- The live day model replans flexible work.
- The constitution prevents bad behavior from becoming a new goal.
- Gmail triage and Calendar work through JARVIS-owned OAuth.
- The user manages connectors from the admin page.
- Read-only finance access works for selected accounts or remains feature-flagged with a clear blocker.
- Every high-impact external action is blocked or approval-gated.
- Every important decision is auditable.
- Web and Telegram recovery paths exist.
- Seven-day soak tests pass for the WhatsApp vertical slice.
- Security, integration, and agent eval suites pass.

29. Research-based implementation decisions

This PRD follows current Codex project conventions:

- Project instructions in `AGENTS.md`.
- Repository skills in `.agents/skills/<skill>/SKILL.md`.
- Project custom agents in `.codex/agents/*.toml`.
- Project MCP configuration in `.codex/config.toml`.

The build kit includes these files so Codex receives consistent instructions before implementation.

Current external research also supports the following decisions:

- Evolution API supports Baileys and the official WhatsApp Cloud API, but active GitHub issues justify reconciliation and version pinning.
- Google Gmail watches require renewal at least every seven days, with daily renewal recommended, and push notifications need fallback synchronization.
- Google Calendar channels do not renew automatically.
- Plaid's current Trial plan supports up to ten production items for eligible new US and Canada teams and includes Transactions and Liabilities.
- WHOOP recommends quick webhook responses, signature validation, asynchronous processing, and reconciliation.
- HealthKit requires user authorization and a native app for background delivery.
- Railway private networking keeps inter-service traffic off the public internet.
- pg-boss provides Postgres-backed scheduling and retries, while application handlers still need idempotency for external side effects.

`Seedocs/RESEARCH_NOTES.md` for source links, dates, limitations, and unresolved findings.

30. Final product standard

JARVIS must remember what matters, notice what the user misses, protect important commitments, adjust plans without losing the mission, keep communication useful, and earn trust through restraint and evidence.

The strongest version is not the one with the longest prompt. It is the one with clean state, strict permissions, durable scheduling, source-backed memory, reliable connector synchronization, conflict handling, measured learning, and a recovery path for every external dependency.

Appendix A: Research and source notes

Research date: August 23, 2026

This file records the sources used to choose the build structure. Features, prices, policies, and releases change. Codex must verify current official documentation before implementing a version-sensitive feature.

Codex project structure

AGENTS.md

Source: <https://developers.openai.com/codex/guides/agents-md>

Finding: Codex reads `AGENTS.md` before work. It builds an instruction chain from the repository root toward the current directory. The default combined project instruction limit is 32 KiB.

Decision: Keep the root `AGENTS.md` concise. Put detailed workflows in skills and docs.

Skills

Source: <https://developers.openai.com/codex/skills>

Finding: A skill is a folder containing `SKILL.md` with `requiredNameAndDescription`. Repository skills are loaded from `.agents/skills`.

Decision: Include narrow project skills for WhatsApp, connectors, security, data, deployment, and evals.

Custom agents

Source: <https://developers.openai.com/codex/agent-configuration/subagents>

Finding: Project custom agents live in `.codex/agents/*.toml` and require `name`, `description`, and `developer_instructions`. Subagents are useful for parallel exploration and review.

Decision: Include read-only explorer, researcher, architect, security, and eval agents, plus focused implementers. Keep one implementation owner per change.

MCP

Source: <https://developers.openai.com/codex/mcp>

Finding: Trusted projects may define MCP servers in `.codex/config.toml`. Codex supports Streamable HTTP and STDIO, OAuth, tool allowlists, and approval modes.

Decision: Configure official documentation and optional infrastructure MCPs. Prompt for write operations.

OpenAI models

Sources:

- <https://developers.openai.com/api/docs/guides/latest-model>
- <https://developers.openai.com/api/docs/models/gpt-5.3-codex>

Finding: GPT-5.6 uses Sol, Terra, and Luna routes. GPT-5.3-Codex is the current agentic coding model in the published model catalog at research time.

Decision: Runtime model routes are environment-configurable. Codex custom agents mostly inherit the parent model to avoid stale hardcoding. Read-only research agents may use Luna or Terra.

Evolution API

Sources:

- <https://github.com/evolution-foundation/evolution-api>
- <https://github.com/evolution-foundation/evolution-api/releases>
- <https://github.com/evolution-foundation/evolution-api/issues/2110>
- <https://github.com/evolution-foundation/evolution-api/issues/2647>
- <https://github.com/evolution-foundation/evolution-api/issues/2326>
- <https://github.com/evolution-foundation/evolution-api/issues/2270>

Findings:

- Evolution API supports a Baileys-based WhatsApp Web connection and official WhatsApp Cloud API.
- The repository lists stable 2.3.7 as the latest stable release, while 2.4.0 release candidates were marked for validation and included licensing changes.
- Open issues report unique messages incorrectly discarded as duplicates, inbound messages missing webhook emission, state degradation, and LID identifier changes.

Decision:

- Use a dedicated number.
- Pin an exact stable image and digest after a staging spike.
- Never use `latest`.
- Start with one replica and Redis disabled.
- Add independent idempotency, reconciliation, health monitoring, and fallback channels.
- Treat Evolution as replaceable transport.

Railway

Sources:

- <https://docs.railway.com/networking/private-networking>
- <https://docs.railway.com/volumes>
- <https://docs.railway.com/volumes/backups>

Findings:

- Services in one environment communicate through private internal DNS.
- Volumes provide persistent service storage.
- Volume backup features are available according to plan and configuration.

Decision: Run API, worker, Evolution, and Evolution data services in one Railway project with private networking. Attach the Evolution session volume and test restoration.

Vercel

Sources:

- <https://vercel.com/docs/monorepos>
- <https://vercel.com/docs/frameworks/full-stack/nextjs>

Finding: Vercel supports monorepo projects and first-class Next.js deployment.

Decision: Deploy only apps/web to Vercel. Keep the durable worker on Railway.

Neon

Sources:

- <https://neon.com/docs/connect/connection-pooling>
- <https://neon.com/docs/introduction/branching>
- <https://neon.com/docs/extensions/pgvector>

Findings: Neon provides pooled Postgres connections, branching, and pgvector support.

Decision: Use a separate Neon project as canonical brain storage. Use pooled application connections, branches for tests, pgvector plus full-text search, and sanitized preview data.

Background jobs

Source: <https://github.com/timgit/pg-boss>

Finding: pg-boss provides Postgres-backed jobs, cron scheduling, retries, concurrency controls, and dead-letter behavior.

Decision: Use pg-boss to avoid adding another queue system in V1. Keep handlers idempotent for external side effects.

Google Gmail

Sources:

- <https://developers.google.com/workspace/gmail/api/guides/push>

- <https://developers.google.com/workspace/gmail/api/guides/sync>

Findings:

- Gmail push uses Cloud Pub/Sub.
- Mailbox watches must be renewed at least every seven days, with daily renewal recommended.
- Notifications contain a history ID and require `history.list` to fetch changes.
- Notifications may be delayed or dropped, so periodic synchronization is required.

Decision: Use read-only Gmail, daily watch renewal, durable history cursor, and reconciliation.

Google Calendar

Source: <https://developers.google.com/workspace/calendar/api/guides/push>

Findings:

- Calendar sends HTTPS notification headers, not changed event bodies.
- Notification channels expire and do not renew automatically.
- The application should use a channel token and fetch actual changes.

Decision: Renew channels before expiration, store sync tokens, verify channel identity, and deduplicate event updates.

Plaid

Sources:

- <https://support.plaid.com/hc/en-us/articles/16194695660311-Can-I-use-Plaid-for-free>
- <https://plaid.com/docs/account/billing/>
- <https://support.plaid.com/hc/en-us/articles/39994173227159-What-is-the-Plaid-Trial-plan>
- <https://plaid.com/docs/link/>
- <https://plaid.com/docs/transactions/>
- <https://plaid.com/docs/liabilities/>
- <https://plaid.com/docs/api/webhooks/webhook-verification/>

Findings:

- Eligible new US and Canada Plaid teams created on or after April 15, 2026 may use a free Trial plan with up to ten production Items.
- The Trial bundle includes Transactions and Liabilities, and most major OAuth institutions including Chase.
- Pay-as-you-go has no minimum commitment. Exact rates are visible in the Plaid Dashboard.
- Plaid webhook verification uses signed headers.

Decision: Use Plaid as the first read-only provider behind an adapter. Confirm Mercury coverage during setup. Never request Transfer.

WHOOOP

Sources:

- <https://developer.whoop.com/docs/developing/oauth/>
- <https://developer.whoop.com/docs/developing/webhooks/>

Findings:

- Offline scope provides refresh-token access.
- v2 webhooks use UUID record IDs.
- WHOOP retries failed webhook delivery five times over about one hour.
- WHOOP recommends a successful response within one second, signature validation, asynchronous processing, and reconciliation.

Decision: Use v2, validate raw-body signatures, enqueue, acknowledge quickly, fetch the referenced record, and reconcile.

Apple Health

Sources:

- <https://developer.apple.com/documentation/healthkit>
- <https://developer.apple.com/documentation/healthkit/executing-observer-queries>

- [https://developer.apple.com/documentation/healthkit/hkhealthstore/enablebackgrounddelivery\(for:frequency:withcompletion:\)](https://developer.apple.com/documentation/healthkit/hkhealthstore/enablebackgrounddelivery(for:frequency:withcompletion:))

Findings:

- HealthKit is a device repository requiring user authorization.
- Observer queries monitor changes.
- Background delivery requires the native entitlement and device testing.
- Access may be limited or denied, so missing data must not be interpreted as zero.

Decision: Build HealthKit only in the future native iOS app. Upload normalized deltas and permission freshness.

Authentication

Source: <https://better-auth.com/docs/plugins/passkey>

Finding: Better Auth provides a passkey plugin based on WebAuthn and FIDO2.

Decision: Use passkey-first single-user authentication with public registration disabled after bootstrap.

Community and issue research boundaries

GitHub issues were used to identify operational risks, not to establish guarantees. Open issues may be environment-specific, outdated, or unresolved. Every version-sensitive decision requires a staging test against the exact deployed release.

Appendix B: Codex build-kit map

The companion ZIP contains the project instructions, custom agents, repository skills, runtime prompts, phase prompts, schemas, ADRs, deployment notes, evaluation cases, and starter configuration required to begin implementation in Codex.

Folder or file	Purpose
AGENTS.md	Root Codex instructions and non-negotiable product rules.
.codex/agents	Narrow custom subagents for exploration, planning, implementation, review, and tests.
.agents/skills	Repository workflows for product behavior, connectors, security, database, evaluations, Evolution, and deployment.
prompts/runtime	Versionable production prompt modes for the JARVIS orchestrator.
prompts/codex	Ordered implementation prompts from bootstrap through production hardening.
schemas	Strict contracts for events, decisions, memories, approvals, connectors, and day plans.
docs/ADR	Accepted architecture decisions.
templates	Environment, connector, Docker, and Railway deployment starting points.